

Fase 1 · Qué hicimos, explicado sin tecnicismos

Para Dirección · 11 de septiembre de 2026 · ARLES RELAY I, versión objetivo 1.2.0

Este documento cuenta lo mismo que FASE-01-CIMIENTOS.md, pero sin dar por sabido nada de programación. Se lee en diez minutos.

1. En una frase

La Fase 1 no construyó la aplicación: construyó los cimientos y las reglas que impiden que la aplicación se rompa más adelante.

Todavía no hay pantallas que un usuario pueda usar. Lo que hay es el terreno preparado, las tuberías puestas y las alarmas instaladas. Es la fase que nadie ve y la que decide si el resto del proyecto va a ir rápido o va a ir corrigiendo errores.

2. La idea que hay que llevarse de aquí

En un proyecto de software, la mayoría de los desastres no ocurren porque alguien escriba algo mal hoy. Ocurren porque **una decisión buena que se tomó al principio se olvida seis meses después** y alguien, sin mala intención, hace lo contrario.

La Fase 1 se ha dedicado a convertir las decisiones importantes en **reglas automáticas**: comprobaciones que corren solas cada vez que alguien toca el proyecto, y que **impiden** que el trabajo avance si se ha roto una de esas reglas.

Un ejemplo concreto. Decidimos que la aplicación **nunca** diga «entregado» cuando lo único que sabemos es que el proveedor de correo aceptó el mensaje. Eso podría haberse quedado en una nota en un documento. En vez de eso, hay una comprobación automática que revisa todos los textos de la aplicación y **detiene el proyecto** si alguno afirma una entrega. La decisión ya no depende de que alguien se acuerde.

Hoy hay **32 comprobaciones de este tipo**. Las 32 pasan. Ninguna está desactivada ni omitida.

3. Qué se construyó, en lenguaje llano

Piense en la aplicación como un edificio de cuatro plantas, construidas de abajo arriba:

	Qué es	Para qué sirve
Las reglas del negocio	Qué es un contacto, qué es una campaña, cuándo un envío está hecho y cuándo no	Es la parte que decide. No toca internet ni archivos, así que se puede comprobar entera, en segundos, sin montar nada

	Qué es	Para qué sirve
La caja fuerte de datos	Dónde se guardan contactos, campañas y el historial	Guarda la información cifrada : el archivo en disco es ilegible sin la llave
La aplicación de escritorio	El programa que se instala en Windows y macOS	Está configurado para no poder hacer cosas que no necesita: no ejecuta programas, no navega por los archivos del equipo, no llama a internet por su cuenta
La interfaz	Lo que el usuario ve	Por ahora es el esqueleto: colores, tipografía, navegación y textos, sin funcionalidad real

Y, aparte del edificio, tres herramientas propias:

- Un generador que convierte **una sola lista de colores** en todo lo demás (los estilos, la lámina de la paleta, los iconos de la aplicación). Cambiar un color se hace en un sitio y se propaga solo.
- Un verificador de **contraste**: comprueba que los textos se lean bien, según el estándar internacional de accesibilidad. No es opinión: es un cálculo.
- Un **validador de fase**: el que ejecuta las 32 comprobaciones y emite el informe.

4. Las tres promesas que ya están garantizadas por el sistema, no por confianza

4.1 Nadie recibe dos veces el mismo correo

Es la promesa más delicada del producto: un correo duplicado quema la reputación del remitente y **no se puede deshacer**.

No lo resolvimos con una revisión antes de enviar —eso falla si el programa se cae en mitad de la campaña—. Lo resolvimos haciendo que **la base de datos sea físicamente incapaz** de aceptar dos envíos a la misma dirección en la misma campaña. Aunque alguien lo intentara a propósito, el sistema lo rechaza.

El caso difícil, y la decisión que tomamos. Si el equipo se apaga justo después de que el proveedor aceptó el correo pero antes de que quedara anotado, nadie sabe con certeza si salió. La regla que fijamos: **ante la duda, no se reenvía**. Se marca como «probablemente enviado», se le muestra al usuario y él decide. Preferimos un correo que quizá no salió a un correo que llega dos veces.

4.2 Los datos personales están cifrados de verdad

No basta con decir que usamos cifrado. Lo comprobamos así: guardamos un nombre reconocible, cerramos la aplicación, **abrimos el archivo en crudo** y verificamos que ese nombre no aparece por ningún lado. Después intentamos abrirlo con una llave equivocada y exigimos que falle.

La llave vive en el almacén de seguridad del sistema operativo (el mismo que guarda las contraseñas del equipo), nunca en un archivo del proyecto. **Si ese almacén no está disponible, la aplicación se niega a arrancar** en vez de funcionar sin cifrado. Es una molestia deliberada.

4.3 Borrar un contacto no borra la prueba de lo que se le envió

Si la empresa recibe una petición de baja y borra un contacto, la ley obliga a eliminar sus datos personales. Pero el registro de «el 3 de marzo se le envió esta campaña» **tiene que sobrevivir**: es la única prueba de lo que hizo la empresa.

Ahora se conserva el hecho y se elimina la identidad. Y tiene un efecto práctico que descubrimos revisando: si alguien vuelve a importar a esa persona en una lista nueva, **el sistema sigue sabiendo que ya se le envió y no le vuelve a escribir**.

5. Lo que encontramos al revisar (y por qué es una buena noticia)

Al terminar la fase, en lugar de darla por buena, la revisamos entera. **Encontramos 13 defectos. Los 13 están corregidos.**

Lo relevante no es el número, sino qué tipo de defectos eran: **las comprobaciones automáticas pasaban mientras la promesa estaba rota**. Dos ejemplos:

- **Borrar un contacto destruía el registro de sus envíos.** Todo funcionaba en apariencia. Solo se ve cuando alguien se sienta a preguntar «¿y qué pasa exactamente si se borra un contacto?».
- **Si el equipo perdía la llave del cifrado** —una reinstalación, un cambio de ordenador— la aplicación generaba una llave nueva sin avisar. El usuario veía un error genérico y no se enteraba de que sus datos seguían ahí y de que lo único que los recupera es una copia de seguridad. Ahora lo dice con claridad.

Tres de los 13 defectos **estaban en las propias comprobaciones**, no en el producto: comprobaciones que no comprobaban nada y siempre daban el visto bueno.

De ahí sale la regla de trabajo que adoptamos: **una alarma que nunca se ha visto sonar no es una alarma**. Antes de dar una comprobación por buena, rompemos algo a propósito y verificamos que la detecta.

6. Lo que todavía NO está comprobado

Esta sección importa más que la anterior, y está declarada a propósito.

Qué falta comprobar	Por qué	Cuándo
Ver la aplicación abrirse en pantalla	El servidor donde se construye no tiene pantalla. El programa compila y arranca, pero nadie lo ha visto abrir una ventana	Fase 2, en un equipo con monitor
Que se vea igual en Windows y en macOS	Cada sistema dibuja la interfaz con un motor distinto. Es un riesgo conocido y aceptado	Fase 2, pantalla por pantalla
Que el instalador funcione	Necesita los certificados de firma, que aún no tenemos	Fase 9
El rendimiento con 500 000 contactos	La estructura está preparada, pero no se ha medido con volumen real	Fase 4

Qué falta comprobar	Por qué	Cuándo
El almacén de contraseñas de Windows y macOS	Aquí solo se probó el equivalente de Linux	Con la primera construcción en los tres sistemas

Y una limitación honesta del método: las comprobaciones automáticas verifican **que las reglas siguen en su sitio**, no que las reglas sean las correctas. Eso lo decide una persona revisando, como se hizo en esta fase.

7. Lo que necesita a Dirección

Tres asuntos. **Ninguno bloquea el trabajo ahora mismo**, pero los tres tienen plazos externos que no controlamos, y por eso conviene arrancarlos pronto.

Asunto	Qué hace falta	Para cuándo
Licencia de la tipografía Mont	La licencia habitual permite usar la fuente en documentos y en una web, pero no incluirla dentro de una aplicación que se distribuye . Eso requiere una licencia distinta, de pago aparte. Seguimos desarrollando con ella; hay que resolverlo antes de la demo	Antes de la demo
Verificación de Google	Para que ARLES envíe desde cuentas de Gmail, Google exige una revisión que tarda semanas e implica tener un dominio verificado y un aviso de privacidad público. Es el plazo externo más largo del proyecto	Trámite ya arrancado
Certificados de firma	Sin ellos, Windows y macOS muestran una advertencia de seguridad al instalar. Se compran, tardan entre 1 y 3 semanas	Antes de la Fase 9

8. Cómo se comprueba todo esto sin creernos nada

Cualquiera con el proyecto delante ejecuta **una sola orden** y obtiene el informe completo:

```
python3 herramientas/validar/validar.py --fase 1
```

Resultado esperado: **32 pasan, 0 fallan, 0 omitidos**.

Cada comprobación explica **qué verifica y por qué importa**, de forma que un fallo se entienda sin abrir el código. Y las mismas 32 comprobaciones se ejecutan solas cada vez que alguien sube un cambio al proyecto: si una falla, el cambio no entra.

9. Conclusión

La Fase 1 está **cerrada y revisada**. Lo que deja:

- Cimientos contruidos y comprobados, con 32 verificaciones automáticas que pasan.
- Las tres promesas centrales del producto —sin duplicados, datos cifrados, historial que sobrevive al borrado— garantizadas por el diseño del sistema, no por la disciplina de quien lo use.
- 13 defectos encontrados y corregidos **antes** de construir encima de ellos, que es cuando salen baratos.
- Una lista escrita y honesta de lo que aún no se ha comprobado, con fecha para cada punto.

Lo siguiente es la Fase 2: el sistema de diseño, es decir, convertir el esqueleto de la interfaz en las pantallas que el usuario verá de verdad.

TELEMETRY INSIGHT · ARLES RELAY I